



LeetCode 1431: Kids With the Greatest Number of Candies

1. Problem Title & Link

Title: LeetCode 1431: Kids With the Greatest Number of Candies

Link: <https://leetcode.com/problems/kids-with-the-greatest-number-of-candies/>

2. Problem Statement (Short Summary)

There are n kids, and each kid has some candies.

You are given:

`candies[i]`

= number of candies the i -th kid has.

You are also given:

`extraCandies`

For each kid, determine whether giving **all extra candies** to that kid would make them have the greatest number of candies among all kids.

Return a boolean array.

Example

Input:

`candies = [2,3,5,1,3]`

`extraCandies = 3`

Output:

`[true,true,true,false,true]`

Explanation:

Kid 1: $2+3 = 5$ ✓

Kid 2: $3+3 = 6$ ✓

Kid 3: $5+3 = 8$ ✓

Kid 4: $1+3 = 4$ ✗

Kid 5: $3+3 = 6$ ✓

3. Examples (Input → Output)

Example 1



Input:

candies = [2,3,5,1,3]

extraCandies = 3

Output:

[true,true,true,false,true]

Example 2

Input:

candies = [4,2,1,1,2]

extraCandies = 1

Output:

[true,false,false,false,false]

Example 3

Input:

candies = [12,1,12]

extraCandies = 10

Output:

[true,false,true]

4. Constraints

$n == \text{candies.length}$

$2 \leq n \leq 100$

$1 \leq \text{candies}[i] \leq 100$

$1 \leq \text{extraCandies} \leq 50$

Important:

Every kid gets evaluated independently.

5. Core Concept (Pattern / Topic)

Array Traversal

Secondary Topics:



- Maximum Element
- Simulation

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

For each kid:

current candies

+

extraCandies

Check if that value is:

$\geq$ maximum candies currently present

If yes:

true

Otherwise:

false

Example

Input:

candies = [2,3,5,1,3]

extraCandies = 3

Maximum candies:

5

Kid 1:

$2 + 3 = 5$

Can reach maximum.

true

Kid 2:

$3 + 3 = 6$

Greater than maximum.

true

Kid 4:

$1 + 3 = 4$

Less than:

5

Result:

false

Key Observation

We only need one value:

Maximum candies in array

After finding it once, we can evaluate every kid.

Key Insight

Condition:

$\text{candies}[i] + \text{extraCandies} \geq \text{maxCandy}$

If true:

append True

Otherwise:

append False

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
[2, 3, 5, 1, 3]
```

```
extra = 3
```

```
Maximum:
```

```
5
```

```
Check:
```

```
2+3 = 5 ✓
```

```
3+3 = 6 ✓
```

```
5+3 = 8 ✓
```

```
1+3 = 4 ✗
```

```
3+3 = 6 ✓
```

```
Result:
```

```
[T, T, T, F, T]
```

8. Pseudocode (Language Independent)

```
maxCandy = maximum value
```

```
answer = []
```



```
for candy in candies

    if candy + extraCandies >= maxCandy

        answer.add(true)

    else

        answer.add(false)

return answer
```

9. Code Implementation

Python

```
class Solution:
    def kidsWithCandies(
        self,
        candies,
        extraCandies
    ):

        maxCandy = max(candies)

        result = []

        for candy in candies:

            result.append(
                candy + extraCandies >= maxCandy
            )

        return result
```

Java

```
import java.util.*;

class Solution {

    public List<Boolean> kidsWithCandies(
        int[] candies,
        int extraCandies) {

        int maxCandy = 0;

        for(int candy : candies)
            maxCandy =
                Math.max(maxCandy,
```



```
        candy);  
  
        List<Boolean> result =  
            new ArrayList<>();  
  
        for(int candy : candies) {  
  
            result.add(  
                candy + extraCandies  
                >= maxCandy  
            );  
        }  
  
        return result;  
    }  
}
```

10. Time & Space Complexity

Time Complexity

Finding maximum:

$O(n)$

Checking all kids:

$O(n)$

Overall:

$O(n)$

Space Complexity

$O(n)$

For output array.

11. Common Mistakes / Edge Cases

Mistake 1

Updating the maximum after giving candies.

The problem asks:

Compare against original maximum.

Mistake 2

Using:

$>$

instead of:

$>=$



Example:

$$2 + 3 = 5$$

Equal is allowed.

Mistake 3

Actually modifying the array.

Not required.

Edge Cases

Already Maximum

Input:

[5]

Output:

[True]

Everyone Can Reach

Input:

[1,1,1]

extra = 10

Output:

[True,True,True]

Nobody Except Max

Input:

[10,1,1]

extra = 2

Output:

[True,False,False]

12. Detailed Dry Run

Input:

candies = [2,3,5,1,3]

extraCandies = 3

Maximum:

5

Kid	Candies	After Extra	≥ 5 ?	Result
1	2	5	Yes	TRUE
2	3	6	Yes	TRUE
3	5	8	Yes	TRUE
4	1	4	No	FALSE
5	3	6	Yes	TRUE

Return:

[True,True,True,False,True]

13. Common Use Cases (Real-Life / Interview)

Ranking Systems

Can someone reach first place after a bonus?

Gaming

Can a player become top-ranked with extra points?

Resource Allocation

Can a user become the highest holder after receiving resources?

Interview Purpose

Tests:

Maximum Element

Array Traversal

Condition Checking

14. Common Traps (Important!)

Trap 1

Recomputing maximum for every kid.

That becomes:

$O(n^2)$

Trap 2

Using:

>

instead of:

>=

Trap 3

Actually adding candies permanently.

Each kid is evaluated independently.

15. Builds To (Related LeetCode Problems)

LC 1431 - Kids With Greatest Candies

↓

LC 747 - Largest Number At Least Twice of Others

↓

LC 414 - Third Maximum Number

↓

LC 169 - Majority Element

↓

LC 53 - Maximum Subarray

These strengthen maximum-element patterns.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Recompute Max Each Time	$O(n^2)$	$O(n)$	Poor
Precompute Maximum	$O(n)$	$O(n)$	Best

Approach 1: Brute Force

For every kid:

Find maximum again.

Very inefficient.

Approach 2: Precompute Maximum (Chosen)

Find maximum once.

Then compare every kid.

Optimal.

17. Why This Solution Works (Short Intuition)

The only benchmark that matters is the current maximum number of candies.



If a kid can reach or exceed that value after receiving all extra candies, the answer for that kid is true; otherwise it is false.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the indices of kids who can become greatest.

Follow-Up 2

Return how many extra candies each kid still needs.

Follow-Up 3

Determine the minimum extra candies needed for every kid to become greatest.

Follow-Up 4

Give candies to multiple kids instead of one.

Interview Takeaway

The key pattern is:

Precompute Maximum

+

Single Pass Traversal

Whenever you hear:

Can reach maximum?

Can become largest?

Compare against best value?

think:

Find Maximum Once

Then Compare Everyone

This optimization appears frequently in array-based interview questions.